



// AI RELIABILITY · DECISION TEMPLATE

The RAG Architecture Decision Template

Nine decisions that determine whether your retrieval pipeline actually works — worked through, with tradeoffs and a place to record your call, before you write the first line of code.

How to use this

Most RAG systems don't fail because of the model. They fail because of decisions made in the first week — chunking, embedding choice, retrieval strategy — that nobody revisited once the demo worked. Each of the following nine decisions gets a short framing, the real options with their tradeoffs, a heuristic for picking one, and a line to record what you chose and why.

Go through this before you write the first line of retrieval code, or use it to audit a system you've already built — either way, the value is in writing your actual answer down, not just reading the options.

01

Decide in order

Later decisions assume earlier ones. Chunking shapes what retrieval can even find — don't skip ahead.

02

Write the reasoning down

Not just the choice — the "because." That's what a future teammate (or you, in six months) will actually need.

03

Revisit after your first eval run

Every heuristic here is a starting point. Your own eval set is the only thing that should change your mind later.

The nine decisions

1. Chunking strategy
2. Embedding model choice
3. Vector store / index
4. Retrieval strategy (dense / sparse / hybrid)
5. Query transformation
6. Context assembly
7. Groundedness / citation verification
8. Freshness & re-indexing
9. Evaluation

1 Chunking Strategy

How you split source documents determines what's even possible to retrieve later. Get this wrong and no amount of embedding-model quality fixes it downstream.

- **Fixed-size (e.g. 512 tokens, with overlap)**: simple and predictable, but cuts across semantic boundaries — a sentence can end up split from the section explaining it.
- **Recursive / structure-aware (split on headers, then size limits)**: preserves logical units; more implementation work, meaningfully better retrieval precision.
- **Semantic (group by embedding-similarity shifts)**: highest fidelity to meaning, highest compute cost to build and maintain the index.
- **Document-native (one chunk per clause/section/row)**: best for already-structured documents — contracts, tickets, tabular data; doesn't generalize to prose.

HEURISTIC

Default to recursive/structure-aware unless your source documents are already highly structured — then chunk along their native structure instead of imposing an artificial one.

STRATEGY: SIZE / OVERLAP:

2 Embedding Model Choice

The embedding model defines the geometry your entire retrieval system operates in. Swapping it later means re-indexing everything — not a config change.

- **General-purpose commercial (OpenAI, Voyage)**: strong baseline, zero infra to run, ongoing per-token cost.
- **Open-weight, self-hosted (BGE, E5, GTE)**: no per-query cost at scale, but you own serving infrastructure and latency.
- **Domain-tuned / fine-tuned**: best relevance for specialized vocabulary — legal, medical, internal jargon — but needs labeled data and retraining discipline.
- **Multilingual**: required for multi-language corpora — an easy detail to miss until it's a launch blocker.

HEURISTIC

Default to a strong general-purpose commercial embedding unless a labeled eval set already shows it underperforming on your domain. Don't fine-tune before you can measure the lift.

MODEL: DIMENSIONS:

3 Vector Store / Index

This decision matters less than people think early on, and more than people think at scale. Don't let it become the first infrastructure decision that blocks everything else.

- **Managed (Pinecone, Weaviate Cloud, etc.):** fastest to start, no ops burden, ongoing cost scales with volume.
- **Self-hosted (Qdrant, Milvus):** full control and cost efficiency at scale, real operational ownership.
- **pgvector inside infrastructure you already run:** no new system to operate; index performance ceiling is lower than purpose-built stores at very large scale.

HEURISTIC

Below roughly 100k–500k vectors, pgvector inside infrastructure you already run is often simpler than a dedicated vector database. Don't add a new system before you've actually outgrown the one you have.

STORE: INDEX TYPE:

4 Retrieval Strategy

Dense (embedding-similarity) retrieval alone misses exact-match cases — part numbers, error codes, names — that keyword search catches trivially.

- **Pure dense:** best for conceptual/paraphrased queries, weak on exact identifiers.
- **Pure sparse (BM25):** exact-match strength, weak on paraphrase and conceptual similarity.
- **Hybrid (dense + sparse, fused):** covers both failure modes; more moving parts to tune and evaluate.
- **Hybrid + re-ranker (cross-encoder second pass):** highest precision, added latency per query.

HEURISTIC

If queries ever include exact identifiers — SKUs, ticket numbers, citations — pure dense retrieval will quietly fail on exactly those queries. Hybrid is the safer default, not an optimization to defer.

STRATEGY: RE-RANKER (Y/N):

5 Query Transformation

The user's literal query is often a worse retrieval query than a rewritten one — but transformation is a place complexity likes to sneak in before it's earned.

- **Raw query passthrough:** zero added latency or failure surface; misses queries phrased far from source vocabulary.
- **Query expansion (synonyms, related terms):** cheap improvement for vocabulary mismatch; can dilute precision if overused.
- **HyDE (embed a hypothetical answer instead of the query):** strong for conceptual gaps between question and answer phrasing; an extra model call per query.
- **Multi-query (reformulate several ways, merge results):** best recall improvement; multiplies retrieval cost per request.

HEURISTIC

Start with raw passthrough and add transformation only once your eval set shows a specific class of query failing. This should be evidence-driven, not default-on — it adds both latency and a new failure surface.

APPROACH: TRIGGER CONDITION:

6 Context Assembly

How many chunks, in what order, deduplicated how — this determines what the model actually sees. Getting it wrong drowns a good retrieval result in noise.

- **Fixed top-k:** simple and predictable; the same k rarely fits both narrow and broad queries well.
- **Dynamic k (score-threshold cutoff):** adapts to query difficulty; needs a calibrated threshold, not a guessed one.
- **Deduplication pass before assembly:** prevents near-duplicate chunks from crowding out distinct information.
- **Context budget as % of model's window:** keeps room for instructions and conversation history; requires active token accounting.

HEURISTIC

More retrieved context is not free — each irrelevant chunk is a chance for the model to ground a claim in the wrong place. Tune k against your eval set's groundedness score, not against "does this feel like enough context."

TOP-K: ORDERING:

7 Groundedness / Citation Verification

Whether what your system says is actually supported by what it retrieved is not something retrieval quality alone guarantees — a naive citation check gets this backwards on exactly the cases that matter.

- **No verification (trust the model):** zero added cost; no defense against fabrication.
- **Naive substring-presence check:** catches nothing subtle — and creates false confidence, which is often worse than no check.
- **Negation-aware citation checking:** catches the single most common, most damaging failure: citing a fact the source explicitly says doesn't exist.
- **Full claim-decomposition + entailment checking:** highest rigor, highest implementation and latency cost.

HEURISTIC

A naive substring check is often worse than no check at all. At minimum, a negation-aware check closes the most common and most damaging failure mode for the least implementation cost.

RELATED

This is exactly what Sentinel's `check_groundedness` tool does — negation-aware, deterministic-first, free to clone. mleg.tech/sentinel

APPROACH: BLOCK THRESHOLD:

8 Freshness & Re-indexing

An index that's never revalidated becomes a source of confidently wrong answers as the underlying documents age. Decide this now, not after the first stale-answer incident.

- **Manual reindex on deploy:** simplest to build; index staleness tracks how often you remember, not how often data changes.
- **Scheduled reindex (cron):** predictable staleness bound; wastes work reindexing unchanged documents.
- **Event-driven (webhook on document change):** near-zero staleness window; more integration work with the source system.

HEURISTIC

Event-driven reindexing is the correct end state for anything customer-facing, but scheduled reindexing on a tight interval is a reasonable starting point. The wrong choice is "whenever someone remembers to run it."

TRIGGER: INTERVAL (IF SCHEDULED):

9 Evaluation

Retrieval quality and generation quality are different failure surfaces. Measure them separately, or you can't tell which stage actually broke when something goes wrong.

- **Retrieval-only metrics (recall@k, MRR):** tells you whether the right material was even found — diagnosable and fixable in isolation.
- **Generation-only metrics (groundedness, fluency):** tells you whether the model used what it was given correctly — assumes retrieval already succeeded.
- **End-to-end metrics (task success):** closest to what users experience; hardest to debug back to a root cause alone.
- **A labeled eval set spanning all three:** the actual target state — lets you isolate which stage failed, not just that something did.

HEURISTIC

If you can only build one eval mechanism first, build retrieval-quality measurement. You cannot debug generation quality without first knowing whether the model was given the right material to work with.

APPROACH: EVAL SET SIZE:

Putting it together

These nine decisions aren't independent — chunking shapes what retrieval can find, retrieval shapes what context assembly has to work with, and none of it matters if groundedness verification (decision 7) isn't catching what slips through. Once all nine are filled in, read them back as one paragraph, not nine isolated choices — the places where your answers contradict each other are usually where the real risk is hiding.

Once this is filled in

You now have nine documented decisions with reasoning attached — the thing that's usually missing when a RAG system starts misbehaving six months after launch and nobody remembers why chunking was set up the way it was. Revisit this after your first real eval run; the heuristics here are starting points, not final answers.

```
// FOR DECISION 7 SPECIFICALLY
```

Sentinel — an eval-as-MCP-server trust layer

A working, free reference implementation of negation-aware groundedness checking — the deterministic-first approach referenced in decision 7. Clone it and run it against your own retrieval output.

mleg.tech/sentinel

If you've filled this out and the honest answer is that several decisions were made by default rather than on purpose — or the system already exists and none of this was ever written down — that's usually the actual signal it's time for a proper architecture review rather than another pass at tuning. If that's where you've landed, [reach out](#).